

Frontend Performance Engineering: Optimization at Scale

Mitesh Kumar
Senior Software Engineer at Red Hat



Why Frontend Performance Matters

The Business Impact of Performance



Amazon

1 second delay = 7% reduction in conversions



Google

53% of mobile users abandon sites that take >3 seconds to load



Walmart

100ms delay = 1% drop in sales

Core Web Vitals & Performance Metrics



Largest Contentful Paint (LCP)

LCP measures the point in time when the largest visible content element on the page ap pears, typically a hero image, banner, or large heading. If the LCP time is too long, user s will perceive the website as slow, even if other content loads quickly.

LCP ≤ 2.5 seconds

CLS (Cumulative Layout Shift)

CLS tracks **how much the layout unexpectedly moves around** while loading.

Target: ≤ 0.1

FID → INP (Interaction Metrics)

FID (First Input Delay) measures the time between a user's interaction and the browser's first response. For example, a user clicks a button, but the click response is delayed bec ause the browser is running JavaScript. This metric reflects a website's interaction respo nsiveness.

FID: ≤ 100ms

Additional Important Performance Metrics (Beyond Core Web Vitals)



TTFB (Time to First Byte)

Tracks how long the server takes to respond.. If TTFB (Time To First Byte) is too slow, No matter how optimized your frontend is, everything else becomes slow.. Common issue include slow server code, slow database queries, and using overly heavy back-end framev

Ideal TTFB: < 200ms

FCP (First Contentful Paint)

Measures the time when the first content is rendered on the screen. Unlike LCP which measures the main content, FCP measures the earliest visible progress, such as backgro und color, header, small text, or a skeleton screen. It reduces the time users spend staring at a blank screen and lowers perceived loading time.

Fast FCP : <1.8s

TBT (Total Blocking Time)

Total Blocking Time (TBT) measures the total amount of time that the main thread is bl ocked by JavaScript between First Contentful Paint (FCP) and Time to Interactive (TTI) . A high TBT indicates that too much JavaScript is being executed, React is excessively re-rendering, or an overly heavy framework or library is being used.

Target TBT: < 200ms



Resource Loading & Delivery Optimization

Code Splitting and Lazy Loading



Route-Based Code Splitting

Load only the necessary routes to reduce the initial load time; this is accomplished using dynamic imports which drastically improves the user experience.

Third-Party Script Deferral

Asynchronously load non-critical third-party scripts; these implementations ensure they do not block the rendering of the main page content for faster initial load.

Conditional Loading Strategies

Load features only based on user actions; this optimization improves performance by preventing unnecessary resource loading and thus streamlines user interaction.

Resource Hints and Optimization

Preconnect Optimization

Establish early connections to third-party domains; this reduces latency by initiating connections before resources are actually demanded, as a result increases performance

<link rel="preconnect" href="https://cdn.example.com
">

Preload Optimization

Prioritize loading critical resources; this ensures priority and prompt loading of essential assets, in the end speeding up initial render times, thus improving user experience.

<link rel="preload" href="/img/hero.jpg" as="image">

Prefetch Optimization

Load resources for the next navigation; by pre-emptive loading, it improves the responsiveness of the site and thus improves user satisfaction by reducing perceived load times.

<link rel="prefetch" href="/scripts/next-page.js">

CDN and Edge Optimization Techniques



Geographic Distribution Methods

Reduce latency via edge locations by geographically distributing content, which in turn brings content closer to users and thus speeding up delivery.

Brotli/Gzip Compression Methods

Implement Brotli/Gzip compression to achieve 70-90% size reduction; this compression significantly reduces the size of text assets thus improving loading speed and bandwidth usage.

HTTP/2 & HTTP/3 Advantages

Utilize multiplexing & server push features of HTTP/2 and HTTP/3 protocols; this makes connections faster, improves resource loading efficiency, and thus speeds up web performance.

Edge Caching for Static Assets

Cache static assets at edge locations to speed up static deliveries; this caching significantly reduces latency and server load, thus leading to faster loading times.



Optimizing Assets for Performance

Image Optimization Techniques

1 Modern Image Formats (WebP, AVIF)

Use WebP and AVIF formats which are typically 25-50% smaller than JPEG/PNG, offering significantly reduced file sizes therefore greatly improve load times with no quality sacrifice.

2 Lazy Loading and Performance

Leverage native `loading="lazy"` attribute and Intersection Observer API; this approach defers offscreen image loading until needed, thus improving initial page load times and resource usage.

```
<img src="image.jpg" loading="lazy" alt="Example">
```

3 Responsive Images Implementation

Use `srcset` and `sizes` attributes to serve optimized images based on device; this responsive approach significantly enhances user experience, and optimizes bandwidth usage.

```
<img
 src="hero-800.jpg"
 srcset="
  hero-400.jpg 400w,
  hero-800.jpg 800w,
  hero-1200.jpg 1200w,
  hero-2000.jpg 2000w
 "
 sizes="
  (max-width: 600px) 100vw,
  (max-width: 1200px) 50vw,
  33vw
 "
 alt="Hero Image"/>
```

Font Optimization Strategies

FOIT — Flash of Invisible Text — where users literally see *nothing* for a second or two.

01

Font-Display Property Usage

Utilize `swap`, `optional`, and fallback strategies; these avoid the flash of invisible text by controlling font rendering behavior and enhance perceived load times.

font-display: swap

02

Font Subsetting Technique

Include only necessary characters in font files; this font subsetting reduces file size, optimizes font loading, thus resulting in faster render times.

03

Preloading Critical Fonts

Preload critical fonts with `<link rel ="preload">`; this process reduces FOIT (Flash of Invisible Text) and ensure faster display of text thus improving user experience.

Optimizing Other Assets and Delivery

01

SVG Optimization Techniques

Minify SVGs and remove metadata from SVG files; these processes will reduce file sizes and complexity, thus results in faster rendering and improved performance.

02

Video Optimization Practices

Lazy load videos, use poster images, and provide multiple formats; this ensures optimal video delivery, improves user experience, and reduces initial load times.

03

Asset Delivery Optimization

Utilize Gzip/Brotli for text assets and implement long cache headers; these practices ensure efficient asset delivery and cache utilization thus significantly improves page load times.

```
Cache-Control: public,
max-age=31536000,
immutable
```



Network Call Optimization Techniques

API Request Optimization

Step 1

Request Batching Methodologies

Combine multiple API requests into a single request; this strategy reduces the number of network calls thus decreases latency and load on the server.

Step2

Request Deduplication Techniques

Prevent duplicate API calls using caching or debouncing tactics; this reduces redundant network traffic therefore improves performance and conserves resources.

Step 3

GraphQL for Data Fetching

Use GraphQL to fetch only required data fields; this ensures efficient data retrieval, minimizing data transfer, and thus improving performance and reducing bandwidth usage.



Caching Strategies

Application-Level Caching Tactics

Employ in-memory caching for frequently accessed data; this application-level strategy minimizes database hits and reduces response times thus improving performance and scalability.

HTTP Caching with Headers

Leverage `Cache-Control` headers, `ETags`, and `Last-Modified` attributes; these mechanisms improve caching efficiency, reduce server load, and enhance user experience.

Browser Caching Benefits

Leverage browser cache for API responses; storing responses in the browser reduces network calls and thus speeds up subsequent page loads and enhances responsiveness.





Rendering Performance and Optimization

Critical Rendering Path Optimization

01

Inline Critical CSS Advantages

Inline above-the-fold styles to render essential content quickly; this reduces render-blocking requests, speeding up initial page load and improving perceived performance.

02

Defer Non-Critical CSS Loading

Asynchronously load below-the-fold styles using techniques like loadCSS; this prevents render blocking and accelerates rendering of the page's main content.

```
<link
  rel="stylesheet"
  href="/styles/non-critical.css"
  media="print"
  onload="this.media='all'"
>
```

03

Minify Render-Blocking Resourcess

Minimize JavaScript and stylesheet dependencies that block rendering; this reduces delays, improves the rendering progress, and enhance the user experience.

Layout and Paint Optimization Techniques

01

Avoid Layout Thrashing

Batch DOM reads and writes efficiently; this prevents unnecessary recalculations, reducing layout thrashing to improve rendering rates and performance.

02

CSS Transforms for GPU Acceleration

Use CSS transforms for animations to leverage GPU acceleration; leveraging GPU-accelerated animations offloads processing and creates smoother transitions and improved performance.

03

Reduce Paint Complexity Methods

Simplify CSS selectors and reduce paint complexity to enhance rendering performance; optimized rendering of elements reduces the paint time and improves visual responsiveness.



JavaScript Performance and Optimization

Bundle Optimization



Tree Shaking Benefits

Remove unused code by implementing tree shaking; reducing unnecessary script size improves load times, decreases browser processing, and increases performance.

Code Minification Techniques

Minify code to reduce bundle size, which optimizes bandwidth usage and improves load times; by reducing script size, pages load faster and improve user experience.

Code Splitting by Features

Split code by route, feature, and vendor to optimize loading; this reduces initial bandwidth usage, speeds up first-page loads and improves initial response times.

Runtime Performance Optimizations

Debounce & Throttle Strategies

Limit function execution frequency utilizing throttle or debounce; reducing processing load smooths user interactions, conserving system recourses, and refining performance.

Web Workers Implementation

Offload heavy computations to background threads using Web Workers, preventing UI blocking; this ensures the main thread remains unblocked, and maintains superior user interaction.

RequestAnimationFrame Usage

Synchronize animations with the refresh rate using RequestAnimationFrame; enhancing animation smoothness, reducing stuttering, and improving performance and user interaction.

Framework-Specific Optimizations



React Optimizations

Use `React.memo`, `useMemo`, `useCallback`, and `Suspense` to enhance React performance; these tools optimize component rendering, diminish unnecessary calculations and streamline user experience.

Vue Optimization Tactics

Utilize `v-memo`, computed properties, and `keep-alive` to enhance Vue performance; streamlined rendering and lessened re-computations improve reactivity and promote efficiency.



Measurement Strategies Detailed

Synthetic Monitoring Techniques

01

Use Controlled Environments

Synthetic monitoring measures performance in controlled lab-testing scenarios; it offers consistency, and control over test situations for assessing application effectiveness.

02

Key Tools and Testing Suites

Use tools like Lighthouse, WebPageTest, and Chrome DevTools for performance analysis; these tools isolate factors to improve website speed and dependability.

03

Testing Scenarios

Test on various browsers (Chrome, Firefox, and Safari), under multiple network conditions; which improves user reach and utility, improving the user experience across all platforms.

Real User Monitoring (RUM).

Step 1

Measure User Experience

RUM measures actual user experience across different networks, locations, and devices to deliver real-world performance statistics for improving user experiences.

Step2

Key Metrics and APIs

Utilize Navigation Timing API, Resource Timing API, and Paint Timing API for detailed performance tracking; these measurements assess various facets of user experience from speed to rendering.

Step 3

Tools and Integration

Employ Google Analytics, New Relic Browser, and Datadog RUM for user monitoring; these resources provide critical insights into platform performance, therefore optimizing user experiences.



THANKS

Mitesh Kumar

